Σήματα στο UNIX (1)

- Αποστολή (**kill()**, **raise()**)

Σήματα στο UNIX (1)

- Αποστολή (**kill()**, **raise()**)

  Παράδειγμα:

```
if (kill(pid, SIGUSR1) < 0) {
          perror("kill");
          exit(1);
}
```

Σήματα στο UNIX (1)

- Αποστολή (**kill()**, **raise()**)

  Παράδειγμα:

```
if (kill(pid, SIGUSR1) < 0) {
          perror("kill");
          exit(1);
}
```

- Χειρισμός (**signal()**, με SIG_IGN, SIG_DFL ή handler)

Σήματα στο UNIX (1)

- Αποστολή (**kill()**, **raise()**)

  Παράδειγμα:

```
if (kill(pid, SIGUSR1) < 0) {
        perror("kill");
        exit(1);
}
```

- Χειρισμός (**signal()**, με SIG_IGN, SIG_DFL ή handler)

  Παράδειγμα:

```
void sighandler(int signum)
{
        got_sigusr1 = 1;
}

if (signal(SIGUSR1, sighandler) < 0) {
        perror("could not establish SIGUSR1 handler");
        exit(1);
}
```

Σήματα στο UNIX (2)

Σήματα στο UNIX (2)

- Αναξιόπιστα
  - Τι θα γίνει αν έρθουν πολλά σήματα;
    - Η συνάρτηση χειρισμού θα τρέξει από *1* έως *n* φορές
  - Τι θα γίνει αν το σήμα έρθει ενώ η συνάρτηση χειρισμού εκτελείται;
- Race conditions: αυτό θα δουλέψει;

Σήματα στο UNIX (2)

- Αναξιόπιστα
  - Τι θα γίνει αν έρθουν πολλά σήματα;
    - Η συνάρτηση χειρισμού θα τρέξει από *1* έως *n* φορές
  - Τι θα γίνει αν το σήμα έρθει ενώ η συνάρτηση χειρισμού εκτελείται;
- Race conditions: αυτό θα δουλέψει;

Παράδειγμα:

```
void sighandler(int signum)
{
        got_sigusr1 = 1;
}
. . .
got_sigusr = 0;
while (!got_sigusr1)
        pause(); /* Αναμονή έως ότου ληφθεί κάποιο σήμα */
```

Σήματα στο UNIX (3)

- Η signal() δεν είναι φορητή
- Ο handler ακυρώνεται όταν εκτελείται (System V)
  - και πρέπει να επανεγκατασταθεί
  - ή όχι… BSD. Στο Linux; εξαρτάται… libC vs. kernel
- Καλύτερη, φορητή λύση: **sigaction()**

Σήματα στο UNIX (3)

- Η signal() δεν είναι φορητή
- Ο handler ακυρώνεται όταν εκτελείται (System V)
  - και πρέπει να επανεγκατασταθεί
  - ή όχι… BSD. Στο Linux; εξαρτάται… libC vs. kernel
- Καλύτερη, φορητή λύση: **sigaction()**

Παράδειγμα:

```
struct sigaction sa;
sigset_t sigset;

sa.sa_handler = sigchld_handler;
sa.sa_flags = SA_RESTART;
sigemptyset(&sigset);
sa.sa_mask = sigset;
if (sigaction(SIGCHLD, &sa, NULL) < 0) {
        perror("sigaction");
        exit(1);
}
```

Σήματα στο UNIX (4)

- Χρήσιμες εντολές
  - kill, ps, pstree, killall, grep

Σήματα στο UNIX (4)

- Χρήσιμες εντολές
  - kill, ps, pstree, killall, grep

```
$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT       4) SIGILL
 5) SIGTRAP      6) SIGABRT      7) SIGBUS        8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV      12) SIGUSR2
 . . .
```

```
$ ps –ef | grep vkoukis|grep bash
vkoukis   4277  4276  0 12:17 pts/0    00:00:00 bash
```

```
$ kill -TERM  4277
$ kill -9  4277
$ killall -9  bash
```

Σήματα στο UNIX (4)

- Χρήσιμες εντολές
  - kill, ps, pstree, killall, grep

```
$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT       4) SIGILL
 5) SIGTRAP      6) SIGABRT      7) SIGBUS        8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV      12) SIGUSR2
 . . .
```

```
$ ps –ef | grep vkoukis|grep bash
vkoukis   4277  4276  0 12:17 pts/0    00:00:00 bash
```

```
$ kill -TERM  4277
$ kill -9  4277
$ killall -9  bash
```

- Όλες οι λεπτομέρειες στα manual pages

SIGCHLD

SIGCHLD

- SIGCHLD: ένα παιδί άλλαξε κατάσταση
  - Πέθανε κανονικά
  - τερματίστηκε από σήμα
  - έχει σταματήσει λόγω σήματος (SIGTSTP, SIGSTOP)
- Επιτρέπει στη γονική διεργασία να κάνει waitpid() *ασύγχρονα*, όταν χρειάζεται
  - Κάτι συμβαίνει σε ένα παιδί
  - Ο πατέρας λαμβάνει SIGCHLD
  - Εκτελεί waitpid()
    - Ιδανικά: πολλές φορές, με WNOHANG

Κώδικας: παράδειγμα χειρισμού SIGCHLD

Κώδικας: παράδειγμα χειρισμού SIGCHLD

```
void sigchld_handler(int signum)
{
        pid_t p;
        int status;

        /*
         * Something has happened to one of the children.
         * We use waitpid() with the WUNTRACED flag, instead of wait(), because
         * SIGCHLD may have been received for a stopped, not dead child.
         *
         * A single SIGCHLD may be received if many processes die at the same time.
         * We use waitpid() with the WNOHANG flag in a loop, to make sure all
         * children are taken care of before leaving the handler.
         */

        do {
                p = waitpid(-1, &status, WUNTRACED | WNOHANG);
                if (p < 0) {
                        perror("waitpid");
                        exit(1);
                }
                explain_wait_status(p, status);

                if (WIFEXITED(status) || WIFSIGNALED(status))
                        /* A child has died */
                if (WIFSTOPPED(status))
                        /* A child has stopped due to SIGSTOP/SIGTSTP, etc... */
         } while (p > 0);
}
```